SURFR2: What It Does, How It Was Built, and How to Run It

Oct 20, 2026 · @Panos

Summary

SURFR2 finds small-RNA sequences that differ between two conditions, without needing a reference genome or annotation. It counts every 17-nucleotide subsequence (k-mer) in every sample, removes sequencing artifacts, normalises for library size, tests for differences with edgeR, and merges the significant k-mers back into sequences. Optionally it then maps those sequences to one or more genomes and colours the plots by where they map.

It is a rewrite of SURFR (Kalogeropoulos et al., 2025). SURFR pooled all samples of a condition before counting and compared raw totals. SURFR2 keeps samples separate, which is what makes normalisation, statistics and replicate-aware results possible.

The tool runs on Dardel, in a Singularity container or with modules. It passes 13 synthetic test scenarios with planted ground truth, and it was tested on two public datasets.

What SURFR2 does

A run compares two conditions named in a config file, e.g. infected vs uninfected. There are two analyses:

- **Differential expression (`dea`, default):** sequences significantly higher in *either* condition.
- **Condition-specific (`specific`):** sequences present in one condition and absent from the other.

The steps

1. **Validate.** The config and samplesheet are checked before anything runs. The launcher prints the design, then checks every tool in the chosen environment and records its version.
2. **Per sample** (one SLURM job each, in parallel):
    a. BAM or FASTQ is converted to FASTQ.

   b. For UMI libraries (e.g. QIAseq), reads are trimmed and one read is kept per original molecule.
   c. miRTrace performs quality control and adapter trimming.
   d. Reads containing Illumina adapter, primer, PhiX or configured contaminant sequence are removed.
   e. KMC counts all 17-mers.
3. **Candidates.** k-mers with enough reads in enough samples, selected without looking at the conditions, are extracted from every sample.
4. **Statistics.**
   a. k-mers with an A-rich 3′ end (poly-A artifacts) are dropped.
   b. Counts are normalised to counts per million QC-passed reads (CPM), or with median-of-ratios size factors.
   c. With ⩾ 2 samples per condition, edgeR's quasi-likelihood test runs.
   d. k-mers are called by FDR and fold change (DEA) or by detection and absence (specific).
   e. Called k-mers are merged into sequences with dekupl-mergeTags.
5. **Genome mapping** (optional). Merged sequences are mapped with Bowtie 1 to each configured genome, with up to 1 mismatch.
6. **Report.** Static and interactive plots are drawn per cohort:
   - a k-mer scatter;
   - a volcano plot;
   - a sequence-level scatter, coloured by genome when mapping is on.

What you get

- **The called sequences,** with fold change, FDR, the condition they are higher in, the samples they are found in, and their genome hits.
- **Statistics for every tested k-mer,** so you can see why something was or was not called.
- **Interactive HTML plots:** hover over a point to see its counts in every sample.
- **A Shiny app:** explore the results, click a point for per-sample counts, or search for a known sequence.
- **A run summary and provenance:** tool versions and R session, for reproducibility.

What we built and why

SURFR2 began as three requested changes to SURFR: per-sample k-mer counts, read-depth normalisation, and a config file defining the comparison. It grew into a general-purpose tool for everyday designs, including very small ones. The main decisions:

Area	What SURFR2 does	Why
Counting	Per-sample KMC counts; set operations stay in KMC's binary format	A k-mer × sample matrix of tens of millions of k-mers doesn't fit in memory
Candidate selection	Condition-blind pre-filter	Selecting on one condition, then testing on the same data, gives p-values that are too small
Normalisation	CPM per million QC-passed reads, or median-of-ratios	Raw pooled totals ignored group size and depth; reads (not k-mers) make CPM independent of read length
Statistics	edgeR quasi-likelihood test; descriptive mode for 1 vs 1	Works with 2–3 replicates; single-sample designs are run but flagged exploratory
Overlapping k-mers	Each distinct count profile is tested once	Six identical 17-mers per 22-nt read acted as pseudo-replicates and inflated edgeR's confidence 6-fold
Two analyses	Differential expression (default) and condition-specific	DEA was the priority; specificity answers a different question
Results	Labelled with condition names; samples listed per k-mer	Results read as "higher in liver", not "case"
UMI libraries	One read per (insert, UMI) molecule	PCR duplicates inflate counts, especially in low-input EV libraries
Artifacts	Reads with adapter, primer or PhiX k-mers removed; poly-A 3′ k-mers dropped	Concentrated contamination distorted normalisation 8-fold in testing

Area	What SURFR2 does	Why
Genome mapping	Bowtie 1, ≤ 1 mismatch, per genome; plots coloured by genome	Shows host vs pathogen vs unmapped at a glance
Plots	k-mer scatter, volcano, sequence scatter; interactive HTML; Shiny app	Hover and click give the per-sample counts behind each point
Environments	Container (reproducible) or modules (development), with tool versions recorded	Code changes never need an image rebuild

Problems found and fixed along the way

- **SURFR1 merging bugs.** SURFR1 merged k-mers with their reverse complements on stranded data (`-n` means unstranded in mergeTags). It also labelled each sequence with its *least* abundant k-mer, because mergeTags reads its first value column as a p-value.
- **miRTrace out-of-memory kills.** miRTrace's wrapper gave Java half of the node's memory and ignored Java's exit status, so an out-of-memory kill looked like success. SURFR2 runs Java itself and checks that miRTrace's output is complete.
- **Singularity on Dardel.**
  - Its temporary directory must be node-local, not Lustre.
  - With Dardel's configuration, the site's default bind paths hid the image's `/etc`. SURFR2 passes `--no-mount bind-paths`.
- **Test-design issues** were found and fixed rather than tuned around: an odd-sample-count crash in candidate merging, and a flaky 1-vs-1 expectation.

How it was validated

All 13 synthetic regression scenarios pass. Each generates reads with planted ground truth, runs the complete pipeline and checks the result. The simulated reads include biological noise (about 30% variation between replicates) and Poisson sampling, so the statistics are tested realistically. Run them with `bash tests/run_test.sh`.

Scenario	What it checks
`small_dea`, `small`	3 vs 3 paired: true sequences called in the right direction; a significant but non-specific sequence handled correctly by each analysis
`null`, `null_dea`	No true differences: no false discoveries; p-values not inflated (5.4–6.0% below 0.05)
`single`, `single_dea`	1 vs 1: descriptive mode, flagged exploratory
`multi`, `multi_descriptive`	Two cohorts plus external controls: replication logic
`umi`	Raw QIAseq reads: unique molecules match the truth exactly
`artifacts`	Injected contamination removed read for read; without removal, normalisation broke
`genomes`	Mapping categories and mismatch counts exact; plots coloured correctly
`polya`	Poly-A tail k-mers removed; the sequence body still called

The container was tested end to end on Dardel.

Real datasets

- **Bat liver vs kidney** (PRJNA572574, Bat1K). Nothing was significant. The diagnostic showed replicates from different bat species, so k-mer differences between species swamped the tissue signal. The lesson: SURFR2 needs replicates from one species.
- **Toxoplasma-infected vs uninfected neuronal EVs** (PRJNA1193507, QIAseq libraries, 3 vs 3). The run worked with UMI deduplication, artifact removal and genome mapping to human and *Toxoplasma*. The published marker miR-221-3p served as the positive control.

Before you start

Is your data suitable?

Requirement	Why
Small-RNA sequencing (miRNA-seq, EV small RNA, …)	Built for short inserts of about 18–40 nt
Two conditions	e.g. treated vs untreated
⩾ 2 samples per condition, ideally ⩾ 3	Needed for the statistical test; 1 vs 1 runs but only descriptively
One species	k-mers differ between species; mixed species make replicates disagree
Single-end FASTQ (gzipped or not) or BAM	For paired-end small-RNA data, use read 1

Paired designs (e.g. both conditions from each donor or batch) are supported through a `block` column.

One-time setup on Dardel

1. **Unpack the code** in your project space:

   ```
   P=/cfs/klemming/projects/snic/<your project>/programs
   mkdir -p $P && cd $P && unzip SURFR2.zip
   ```

2. **Let Python read the config** (login node): `python3 -c "import yaml" || pip install --user pyyaml`
3. **Choose an environment.**
   - **Container (recommended):** you need read access to the SURFR2 image (`surfr2_<version>.sif`). Check with `ls -l` on its path.
   - **Modules (development):** install the tools SURFR2 needs that aren't modules, once: `bash $P/SURFR2/container/install_tools.sh $P/surfr2-tools --with-pigz`. Then install the R packages into a personal library: `ml PDC R/<version>` and `Rscript $P/SURFR2/container/install_r_packages.R $P/R-lib`.

To build the container yourself, use Docker on a laptop: `docker buildx build --platform linux/amd64 -f container/Dockerfile -t surfr2:<version> --load container/`. Then convert it there with the Singularity image (`quay.io/singularity/singularity:v4.1.0 build … docker-archive://…`) and copy the `.sif` to Dardel. Don't convert on Dardel with temporary files on Lustre.

Running SURFR2 step by step

Step 1: Identify the library type

Raw small-RNA reads usually still contain the adapter, and some kits add UMIs or random bases. Run the detector on one file. It reads the first 200,000 reads and ends with a line starting `==>` that tells you what to set:

```
bash $P/SURFR2/bin/surfr2_detect_adapter.sh /path/to/one_sample.fastq.gz
```

Verdict	Config
Reads already trimmed	`adapter: null`
TruSeq or NEBNext adapter	`adapter: <the sequence printed>`
QIAseq miRNA	`adapter: AACTGTAGGCACCATCAAT`, `umi_length: 12`
NEXTflex (4 random bases each side)	Not supported yet

Step 2: Write the samplesheet

A tab-separated file, one row per sample. Columns may be in any order:

```
sample_id       condition       file_type       path
ctrl_1  untreated       fastq
/cfs/klemming/projects/snic/<project>/data/ctrl_1.fastq.gz
trt_1   treated fastq
/cfs/klemming/projects/snic/<project>/data/trt_1.fastq.gz
```

- **`sample_id`**: unique; letters, digits, `.`, `_`, `-`.
- **`condition`**: the group name. These names appear in every result.
- **`file_type`**: `fastq` or `bam`.

- **`block`** (optional): for paired designs, e.g. the donor.

Step 3: Write the config

Copy the Dardel template and edit the lines marked `<-- EDIT`:

```
cp $P/SURFR2/config/config.dardel.yaml my_config.yaml
```

A complete minimal config:

```
project: my_project
samplesheet: samples.tsv
outdir: /cfs/klemming/projects/snic/<project>/SURFR2_results/my_project
analysis: dea                          # or: specific

comparison:
  case: treated                        # log2FC = log2(treated / untreated)
  control: untreated

qc:
  adapter: null                        # from step 1
  umi_length: 0                        # 12 for QIAseq

# optional: map results to genomes (Bowtie 1 index prefix, or a FASTA to
index)
# genomes:
#   - {name: human, bowtie_index: /path/to/bowtie1/hg38}
#   - {name: toxoplasma, fasta: /path/to/toxoplasma.fasta}

execution:
  environment: container
  container: /path/to/surfr2_<version>.sif
  modules: [PDC, singularity]
  bind: [/cfs/klemming]
  slurm:
    account: <your allocation>
    sample: {partition: shared, cpus: 8, mem: 16G, time: "04:00:00"}
    matrix: {partition: shared, cpus: 8, mem: 16G, time: "04:00:00"}
    filter: {partition: shared, cpus: 4, mem: 16G, time: "04:00:00"}
    map:    {partition: shared, cpus: 8, mem: 32G, time: "12:00:00"}
```

For the modules environment, replace the first four `execution` lines with:

```
  environment: modules
  modules: [PDC/26.03, samtools/1.20, R/4.5.2-cpeGNU-26.03, java/21.0.1]
  path_prepend: [/cfs/klemming/projects/snic/<project>/programs/surfr2-
tools/bin]
  r_libs: /cfs/klemming/projects/snic/<project>/programs/R-lib
```

Step 4: Dry run

From the login node:

```
bash $P/SURFR2/run_SURFR2.sh -c my_config.yaml --dry-run
```

Nothing is submitted. The dry run:

- checks the config, the samplesheet and every input file;
- prints the design, e.g. `treated=3 untreated=3 [test]`; `[descriptive]` means no test is possible;
- checks every tool in the chosen environment;
- lists the jobs it would submit.

If an unedited `<…>` placeholder is left, it names the line.

Step 5: Run and monitor

```
bash $P/SURFR2/run_SURFR2.sh -c my_config.yaml
squeue -u $USER
```

Run this on the login node, not with `sbatch`: the launcher submits the jobs itself. You get one array job processing all samples in parallel, then short jobs for candidates, statistics, mapping (if configured) and the report. Six samples typically finish within an hour. Building a human genome index adds one to a few hours, once.

Logs are in `<outdir>/runs/latest/logs/`. If anything fails, fix the cause and run the same command again: finished work is reused.

Reading the results

Start with `run_summary.txt`, then the merged sequences. Everything is in `<outdir>/results/`:

File	Content
`run_summary.txt`	Read first: design, settings, counts, artifacts removed, genome mapping, warnings
`de_sequences.tsv` (`specific_sequences.tsv`)	The main result: merged sequences with fold change, p-value, `higher_in`, `found_in_<condition>` and, with genomes, per-genome mismatches, hits, locus and `genome_category`
`de_kmers.tsv`	Every called k-mer with all statistics
`de_kmers_counts.tsv`, `de_kmers_cpm.tsv`	Raw counts and CPM of the called k-mers per sample
`kmer_stats.tsv.gz`	Statistics for every tested k-mer
`genome_mapping.tsv`	Each sequence's best hit per genome
`polya_kmers_removed.tsv.gz`	k-mers dropped for an A-rich 3′ end
`normalisation.tsv`	Reads used, artifact reads removed and size factor per sample
`interactive/scatter_*`, `volcano_*`, `sequences_*.html`	Interactive plots: hover over a point for its counts
`plots/*.pdf`	The same plots for publication, plus QC plots
`tool_versions.txt`, `sessionInfo.txt`	Exact tools, modules and R packages used

Terms. CPM means reads containing the sequence per million QC-passed reads. `log2fc` is log2(case / control): +1 means 2× higher in the case condition. The plot colours are:

- **by genome** when mapped: one colour per genome, purple for both or several genomes, black for unmapped;
- **by the higher condition** otherwise.

Sanity check with a known sequence

Check a sequence that should differ, e.g. a published marker:

```
SINGULARITY_TMPDIR=/tmp SURFR2_CONTROLS="miR-221-3p=AGCTACATTGTCTGCTGGGTTTC"
\
  singularity exec --no-mount bind-paths -B /cfs/klemming
/path/to/surfr2_<version>.sif \
  Rscript $P/SURFR2/bin/surfr2_diagnose.R <outdir>/runs/latest
```

It reports:

- the control's statistics;
- whether replicates agree, and how variable they are (BCV);
- whether samples group by condition;
- how many k-mers are found in only one sample.

If nothing is significant, this usually shows why.

Exploring interactively

Copy `results/` to a laptop with R and the packages `shiny`, `plotly`, `dplyr`, `readr`, `ggplot2` and `jsonlite`, then run:

```
SURFR2_RESULTS=/path/to/results Rscript -e 'shiny::runApp("SURFR2/app",
launch.browser = TRUE)'
```

Switch between the k-mer, volcano and sequence plots, click any point for its per-sample counts, or search for a sequence.

Re-running and changing settings

After changing anything, run the same command again. Each step records the inputs and settings it was computed with: samples and candidate k-mers are recomputed only when those changed, and everything else is reused. Shortcuts skip the sample jobs:

You changed only…	Command
`dea` or `statistics` thresholds, or `artifacts.polya`	`run_SURFR2.sh -c my_config.yaml --from filter`
`genomes` or `mapping` (e.g. added a genome)	`run_SURFR2.sh -c my_config.yaml --from map`

You changed only…	Command
nothing; you just want the plots redrawn	`run_SURFR2.sh -c my_config.yaml --from report`

Don't use `--from filter` after changing `analysis` or the `specific` thresholds: these can change which k-mers are tested, and only a full run picks that up. Changing the adapter, UMI length, artifact references or input files makes the affected samples recompute automatically.

Key configuration options

Every option has a default. `config/config.example.yaml` documents all of them, and the validator rejects unknown keys, so typos fail immediately.

Option	Default	Meaning
`analysis`	dea	`dea` (either direction) or `specific` (present in case, absent in control)
`comparison.case` / `.control`	required	Condition names; log2FC = log2(case / control)
`qc.adapter` / `qc.umi_length`	null / 0	3′ adapter to trim; UMI length (QIAseq: 12)
`normalization.method`	cpm	`median_ratio` when a few very abundant small RNAs dominate one condition
`dea.min_abs_log2fc`	1	Minimum fold change (1 = 2-fold); use 2–3 for 1-vs-1 designs
`dea.min_mean_cpm`	1	Minimum mean CPM in the higher condition
`statistics.fdr`	0.05	False discovery rate
`candidates.min_count`	3	Reads needed in a sample for a k-mer to count as present there
`artifacts.enabled`	true	Remove reads with adapter, primer, PhiX or `extra_fasta` k-mers

Option	Default	Meaning
`artifacts.polya`	on: 5 / 8 of 10	Drop k-mers ending in ⩾ 5 A's or with ⩾ 8 A's in the last 10 bases; disable if adenylated isoforms matter
`genomes`	none	List of `{name, fasta}` or `{name, bowtie_index}` (Bowtie 1 `.ebwt`)
`mapping.max_mismatches`	1	Mismatches allowed when mapping to genomes
`specific.*`	strict	Detection and absence thresholds for the specific analysis
`execution.*`	container	Environment, image, modules, SLURM account and resources

Troubleshooting

Most problems are caught by the dry run, which names the cause.

Symptom	Cause and fix
`CONFIG ERROR: …`	The config or samplesheet has a problem; the message names it, including leftover `<…>` placeholders
`host python3 lacks PyYAML`	`pip install --user pyyaml`
`tool check failed`	A tool is missing in the environment; the log lists what and how to fix it
Container errors about FUSE, mounting or `/etc`	Keep the default `container_options` (`--no-mount bind-paths`). For manual `singularity exec`, add that option and `SINGULARITY_TMPDIR=/tmp`
Jobs `CANCELLED`, a sample log shows `oom_kill`	Raise `mem` under `slurm.sample` and rerun
`miRTrace reported an error`	Usually a wrong adapter; or set `qc.phred_offset: 33`

Symptom	Cause and fix
Very few reads pass QC	Wrong or missing `qc.adapter`; repeat step 1
`no Bowtie 1 index at …`	Wrong prefix, or a Bowtie 2 (`.bt2`) index; give the prefix before `.1.ebwt`, or the FASTA
Run summary says EXPLORATORY	A condition has one sample, so no statistical test was possible
Nothing significant	Run the diagnostic: replicates that disagree (mixed species, batch effects) are the usual cause

If a job fails, its log is in `<outdir>/runs/latest/logs/`.

Limitations and good practice

- **Replicates decide what can be found.** 1 vs 1 cannot be tested. In testing, a 1-vs-1 run called 11 of about 60 unchanged sequences at the default 2-fold threshold; use `dea.min_abs_log2fc: 2` or more and treat the calls as hypotheses.
- **One species per comparison.** Replicates from different species or strains differ at the k-mer level, and that swamps the biological signal.
- **Sequences in only some samples have low power.** A sequence present in 5 of 7 cases and absent from controls gives p ≈ 0.01, which doesn't survive FDR among many tests.
- **Short artifact fragments escape.** Adapter fragments shorter than 17 nt can't be detected. The poly-A rule also removes genuinely adenylated isoforms; disable it when those matter.
- **NEXTflex libraries** (random bases on both sides of the insert) aren't supported yet.
- **Mapped is not the same as of that origin.** Conserved RNA fragments can map to more than one genome. A "both" call is biology, not an error.
- **Always include a positive control** for a first run on new data, e.g. a published marker or a tissue-specific miRNA. Check it with the diagnostic script, and read `run_summary.txt` before interpreting anything.

The repository's `README.md` is the quick-start for new users. `README_SURFR2.md` and the code reference guide document every step in detail.